

Deep Learning Non Supervisionato e Sistemi Generativi

NECKER

8 Aprile 2025

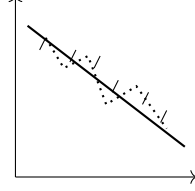
Indice

1	riduzione dimensionalità	2
1.1	Analisi Geometrico-Algoritmica di PCA: Riduzione da 2D a 1D con Collasso del Rumore	2
1.1.1	Passo 1: Centrazione dei Dati e Traslazione dell'Origine	2
1.1.2	Passo 2: Matrice di Covarianza (Σ) e Spessore della Nuvola	3
1.1.3	Passo 3: Estrazione dello Spazio Autovettoriale (Rotazione dei Nuovi Assi)	3
1.1.4	Passo 4: Proiezione Ortogonale e Collasso Dimensionale ($d = 1$)	4
1.2	auto encoder (ae) non lineare	4
1.3	tipi di auto encoder	4
1.3.1	over complete: $K > D$	5
1.3.2	under complete: $K < D$	6
1.4	shallow e deep	6
2	variational autoencoder (vae)	7
2.1	Il Limite degli Autoencoder (AE) Deterministici	7
2.2	La Soluzione VAE: Spessore Probabilistico e Campionamento	8
2.3	La Loss Funzionale: Il Tiro alla Fune Geometrico	8
2.4	Formazione dei Cluster Non Supervisionati e Interpolazione	9
2.5	La Meccanica dell'Aggiornamento dei Pesi e Flusso dei Gradienti	10
2.5.1	Il Grafo Computazionale e il Reparameterization Trick	10
2.5.2	Derivazione Matematica dei Gradienti dei Parametri	10
2.5.3	Il Bilancio Dinamico delle Forze sui Neuroni dell'Encoder	11
3	reti di hopfield	11
3.1	Aggiornamento dei pesi:	11
3.2	Dinamica di Transizione degli Stati Locali	11
3.3	funzione di energia: (vale per le reti simmetriche $\{-1, 1\}$)	12
3.4	apprendimento: (hebbian rule)	13

1 riduzione dimensionalità

Nel Machine Learning e nell'analisi statistica multivariata, la **riduzione di dimensionalità** consiste nel processo di trasformazione di un dataset ad alta dimensionalità $X \in \mathbb{R}^{N \times D}$ in una rappresentazione a dimensionalità inferiore $Z \in \mathbb{R}^{N \times d}$ (con $d \ll D$), preservando al contempo la quota più significativa della struttura informativa originaria.

L'esigenza di tale compressione spaziale risponde alla necessità di mitigare l'aumento delle feature il quale determina una dispersione esponenziale dei punti nello spazio geometrico, rendendo i modelli matematici instabili, computazionalmente onerosi e soggetti a *overfitting*.



1.1 Analisi Geometrico-Algoritmica di PCA: Riduzione da 2D a 1D con Collasso del Rumore

Per comprendere il nesso causale tra le trasformazioni algebriche e il collasso geometrico delle dimensioni superflue, si analizzi la riduzione di dimensionalità di un dataset $X \in \mathbb{R}^{3 \times 2}$ composto da 3 osservazioni non allineate nello spazio cartesiano ($D = 2$), con l'obiettivo di comprimerle su una retta ($d = 1$).

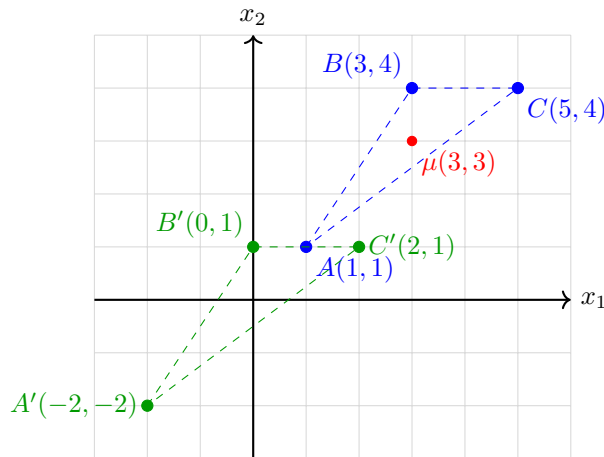
$$X = \begin{bmatrix} 1 & 1 \\ 3 & 4 \\ 5 & 4 \end{bmatrix} \quad (1)$$

1.1.1 Passo 1: Centrazione dei Dati e Traslazione dell'Origine

Si calcola il baricentro geometrico della nuvola tramite il vettore delle medie campionarie $\mu = [\mu_1, \mu_2] = [3, 3]$. Sottraendo μ da ciascuna coordinata, si genera la matrice dei dati centrati $\bar{X}$:

$$\bar{X} = X - \mathbf{1}\mu = \begin{bmatrix} 1-3 & 1-3 \\ 3-3 & 4-3 \\ 5-3 & 4-3 \end{bmatrix} = \begin{bmatrix} -2 & -2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} \quad (2)$$

Significato Geometrico: Questa operazione applica una **traslazione rigida**, spostando il centro di gravità della nuvola dei punti nell'origine degli assi $(0, 0)$. Questo annulla il vettore dei bias e permette alle trasformazioni successive di operare come puro operatore lineare (rotazione).



1.1.2 Passo 2: Matrice di Covarianza (Σ) e Spessore della Nuvola

La matrice di covarianza empirica $\Sigma \in \mathbb{R}^{2 \times 2}$ mappa la dispersione spaziale dei punti centrati:

$$\Sigma = \frac{1}{N-1} \bar{X}^T \bar{X} = \frac{1}{2} \begin{bmatrix} -2 & 0 & 2 \\ -2 & 1 & 1 \end{bmatrix} \begin{bmatrix} -2 & -2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 3 \\ 3 & 3 \end{bmatrix} \quad (3)$$

Significato Geometrico: Gli elementi diagonali indicano che la varianza lungo x_1 ($\Sigma_{11} = 4$) è maggiore rispetto a quella lungo x_2 ($\Sigma_{22} = 3$). La covarianza positiva ($\Sigma_{12} = 3$) indica un trend obliquo. A differenza dell'esempio degenerare, la nuvola possiede uno "spessore" bidimensionale reale: i punti non sono allineati, ma formano un triangolo (un'ellisse di dispersione piena).

1.1.3 Passo 3: Estrazione dello Spazio Autovettoriale (Rotazione dei Nuovi Assi)

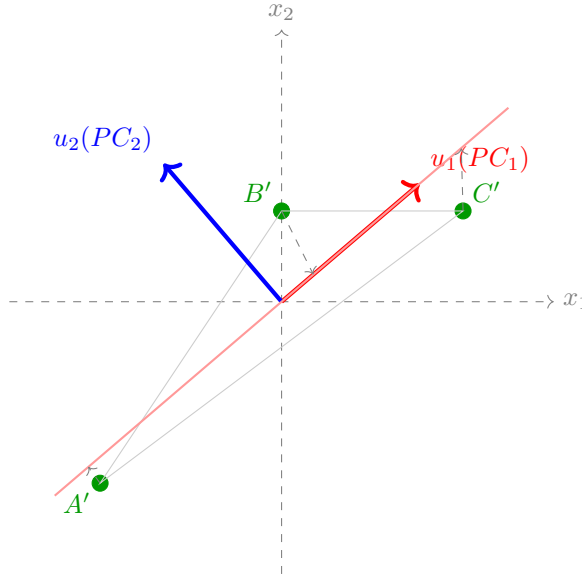
Risolvendo l'equazione caratteristica $\det(\Sigma - \lambda I) = 0$:

$$\det \begin{bmatrix} 4 - \lambda & 3 \\ 3 & 3 - \lambda \end{bmatrix} = 0 \implies \lambda^2 - 7\lambda + 3 = 0 \quad (4)$$

Si ottengono gli autovalori $\lambda_1 \approx 6.54$ (varianza asse maggiore) e $\lambda_2 \approx 0.46$ (varianza asse minore). Calcolando e normalizzando l'autovettore dominante per λ_1 , si ottiene l'asse di proiezione u_1 :

$$(\Sigma - 6.54I)u_1 = 0 \implies u_1 \approx \begin{bmatrix} 0.76 \\ 0.65 \end{bmatrix} \quad (5)$$

Significato Geometrico: Gli autovettori definiscono il nuovo sistema di coordinate ruotato (PC_1, PC_2). L'asse PC_1 (diretto come u_1) si allinea lungo la massima estensione del triangolo dei dati. L'autovalore secondario $\lambda_2 = 0.46$ rappresenta lo spessore ortogonale, inteso come rumore informativo da eliminare durante il collasso dimensionale.

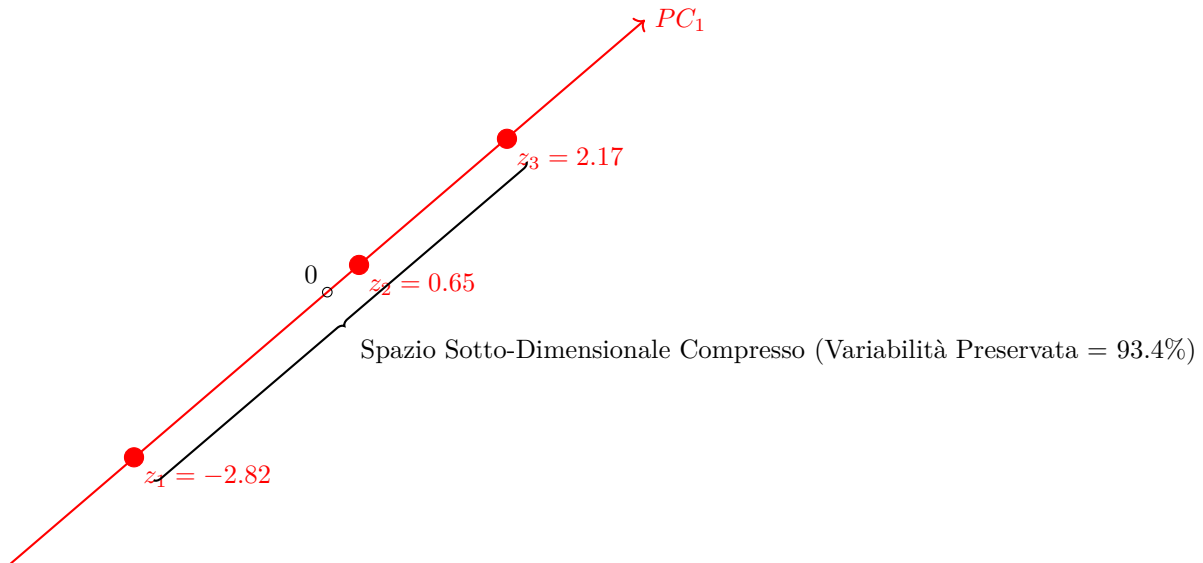


1.1.4 Passo 4: Proiezione Ortogonale e Collasso Dimensionale ($d = 1$)

Impostando la matrice di proiezione $W = [u_1] \in \mathbb{R}^{2 \times 1}$, il mapping lineare schiaccia i dati bidimensionali lungo l'unico asse latente superstite, generando il dataset unidimensionale $Z \in \mathbb{R}^{3 \times 1}$:

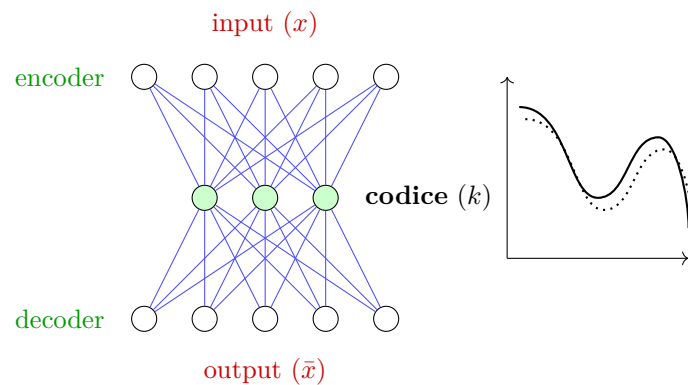
$$Z = \bar{X}W = \begin{bmatrix} -2 & -2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 0.76 \\ 0.65 \end{bmatrix} \approx \begin{bmatrix} -2.82 \\ 0.65 \\ 2.17 \end{bmatrix} \quad (6)$$

Significato Geometrico: Quest'ultimo passaggio esegue una **proiezione ortogonale distruttiva**. I punti A', B', C' , originariamente disposti a triangolo, vengono proiettati ad angolo retto sulla retta PC_1 . Lo "spessore" quantificato da λ_2 viene interamente rimosso (collassato a zero). Il dataset finale non ha più coordinate (x_1, x_2) , ma è composto da semplici scalari che indicano la posizione lineare dei punti lungo il binario PC_1 .



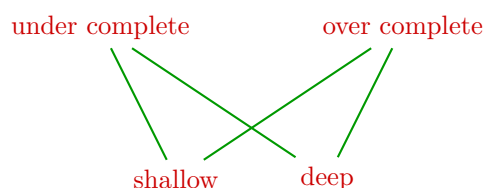
1.2 auto encoder (ae) non lineare

(si usano relu o non lineari)

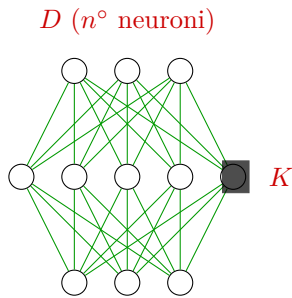


una vera e propria rete neurale il cui scopo è ridurre le feature per poi ricostruirle.
la differenza sostanziale è che è addestrata con l'input. es: $L = |x - \bar{x}|^2$

1.3 tipi di auto encoder



1.3.1 over complete: $K > D$



- utilizzati annullamenti nella generazione
- apprendono rappresentazioni sparse
- o ripuliscono dal rumore

Problema:

nelle reti dove $K < D$ i neuroni sono forzati ad estrarre le feature e a specializzarsi, nelle reti dove $K > D$, la rete generalmente trova la scorciatoia matematica di mettere tutti i pesi a 1 e fare letteralmente copia e incolla, questo fenomeno non permette alla rete di specializzare i neuroni al riconoscimento di determinate caratteristiche. Per ovviare a questo problema, la soluzione è quella di spingere a specializzare solo alcuni neuroni. Questo viene fatto con gli **sparse autoencoders**.

sparse auto encoders:

si aggiunge un termine di correzione nella loss per inibire alcuni neuroni spingendoli ad azzerarsi. La logica consiste nell'utilizzare la somma dell'output di tutti i neuroni moltiplicata per un parametro, e sommarla alla reale funzione di **Loss**. Così facendo la rete dovrà cercare di minimizzare sia la **Loss** iniziale (se è bassa indica che l'immagine è stata ricostruita bene), sia il parametro aggiunto (indicherà che la somma degli output dei neuroni è molto bassa, quindi pochi neuroni attivi). **ATTENZIONE: la Loss iniziale non potrà mai essere negativa, il termine aggiunto non potrà mai essere 0, questo significa che si introduce un piccolo errore e che la rete non avrà mai il 100% di accuratezza.**

$$L = \frac{1}{n} \sum_{i=1}^n |x^i - \bar{x}^i|^2 + \lambda \sum_j |h_j|$$

$\lambda \sum_j |h_j|$:

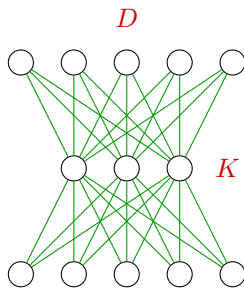
- h_j : output del neurone
- $\sum$: sommatoria di tutti gli output
- λ : iperparametro (come le learning rate) che decide quanto penalizzare

denoising auto encoders:

risolto il problema della specializzazione, incontriamo il problema del denoising, infatti se addestriamo la rete a ricostruire immagini partendo da un input perfetto, nel momento in cui l'immagine fornita è sfocata o imperfetta, il modello avrà difficoltà a ricostruirla in maniera corretta. Per rendere la rete immune a questo problema, si sporca l'input per addestrare la rete a trovare le info dati davvero utili per ricostruire l'input.

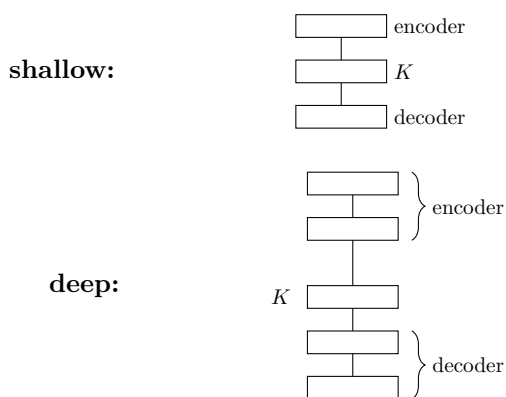
$$X \rightarrow \text{noise } \blacksquare \rightarrow X' \rightarrow \text{encoder} \rightarrow K \rightarrow \text{decoder} \rightarrow \bar{X} \quad \left| \quad L = |X - \bar{X}|^2 \right.$$

1.3.2 under complete: $K < D$



servono a ridurre la dimensionalità estraendo le feature più importanti.

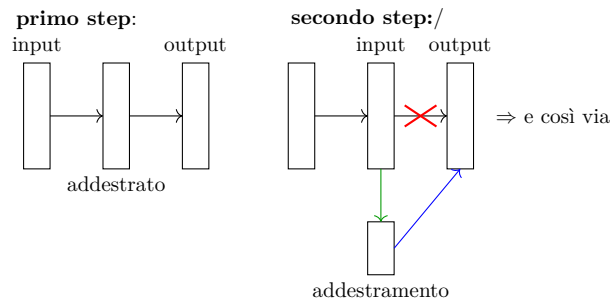
1.4 shallow e deep



apprendono solo qualche trasformazione non lineare.

più profondi (più layers), apprendono caratteristiche molto più complesse (più accurati).

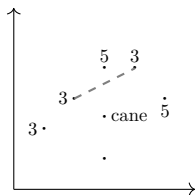
addestramento: ora si addestra come una rete normale curando l'inizializzazione dei pesi per evitare l'explosion o il vanishing gradient. Un tempo invece, la rete veniva addestrata a blocchi:



in pratica si addestrava un blocco e poi si congelavano i pesi e quel blocco diventava l'input di quello aggiunto.

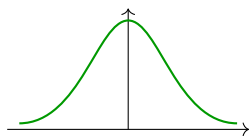
2 variational autoencoder (vae)

gli autoencoder (**vae**) ci permettono di generare meglio lo spazio latente (codice $K \rightarrow$ feature dentro K). Infatti con gli (**ae**) possiamo apprendere per ogni immagine un singolo Z ottenendo spesso rappresentazioni molto diverse per ogni cerchio idoneo.



l'idea è che senza un metodo noto imparo in modo disordinato.

invece attraverso i (**vae**) cerco di istruire la rete a imparare una rappresentazione delle feature intorno allo zero (media intorno a zero e con varianza 1 che fissa il punto).



prior ($P(z)$)

la **prior** indica il modello che deve seguire deve arrivare a una curva simile.

ciò ci permette di poter anche generare immagini diverse spostandoci leggermente (dato che ora tutte le immagini sono ordinate).

$$P(z) \sim \mathcal{N}(\underbrace{0}_{\text{media zero}}, \underbrace{I}_{\text{matrice identità di covarianza}})$$

2.1 Il Limite degli Autoencoder (AE) Deterministici

gli autoencoder (**vae**) ci permettono di generare meglio lo spazio latente (codice $K \rightarrow$ feature dentro K). Infatti con gli (**ae**) possiamo apprendere per ogni immagine un singolo Z ottenendo spesso rappresentazioni molto diverse. Il problema sorge in fase di generazione: campionando un punto nelle vaste regioni vuote tra un dato e l'altro, il decoder riceve un pattern di input casuale e genera puro rumore visivo. Manca la proprietà fondamentale della **continuità**.

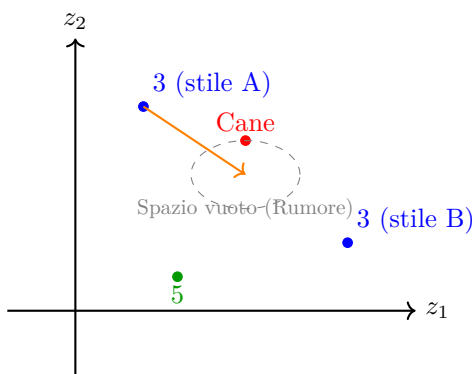


Figura 1: Spazio latente di un AE classico. I dati sono punti isolati separati da vasti spazi vuoti privi di significato generativo.

2.2 La Soluzione VAE: Spessore Probabilistico e Campionamento

Il VAE risolve il problema del vuoto trasformando i punti in **nuvole di probabilità** (distribuzioni gaussiane). L'encoder non sputa più un vettore fisso z , ma due vettori di parametri: la media $\mu(x)$ e la varianza $\sigma^2(x)$.

La distribuzione condizionale dell'hardware latente diventa:

$$q(z|x) \sim \mathcal{N}(\mu(x), \sigma^2(x)I) \quad (7)$$

Per permettere l'addestramento tramite backpropagation, viene introdotto il **Reparameterization Trick**. Il vettore z effettivo dato in pasto al decoder viene campionato aggiungendo un vettore di rumore stocastico $\varepsilon \sim \mathcal{N}(0, I)$:

$$z = \mu(x) + \sigma(x) \odot \varepsilon \quad (8)$$

Significato geometrico: La rete è costretta a ricostruire l'input partendo da un punto a caso *all'interno* della nuvola. Questo forzerà le nuvole ad allargarsi e a riempire lo spazio vuoto, garantendo che ogni punto nell'intorno della media mantenga un significato semantico valido per il decoder.

2.3 La Loss Funzionale: Il Tiro alla Fune Geometrico

L'obiettivo ingegneristico del VAE non è creare tante distribuzioni disordinate, ma forzare l'intero spazio latente a organizzarsi in un **continuum topologico** governato da una Gaussiana standard (Prior) $P(z) \sim \mathcal{N}(0, I)$.

La Loss function del VAE è composta da due forze in competizione:

$$\mathcal{L}_{\text{VAE}} = \underbrace{-\mathbb{E}_{z \sim q(z|x)} [\log p(x|z)]}_{\text{Termine di Ricostruzione}} + \underbrace{\mathcal{D}_{\text{KL}}(q(z|x) \parallel \mathcal{N}(0, I))}_{\text{Divergenza di Kullback-Leibler}}$$

Anatomia della Loss Funzionale

Per comprendere la dinamica di ottimizzazione del VAE, occorre isolare i singoli operatori matematici che compongono il funzionale di costo:

- **$\mathcal{L}_{\text{VAE}}$ (Loss Complessiva):** Rappresenta la funzione obiettivo (*Evidence Lower Bound*, o ELBO, riscritta in ottica di minimizzazione). L'algoritmo applica la discesa del gradiente stocastico per contrarre questo valore numerico globale verso il limite inferiore asintotico.
- $-\mathbb{E}_{z \sim q(z|x)} [\log p(x|z)]$ (**Log-Verosimiglianza Attesa di Ricostruzione**):
 - **Il Segno Meno (-):** il log risulterà negativo, noi sappiamo che se la **Loss** è alta allora il modello sbaglia molto, allora si aggiunge il segno meno per rendere positivo il tutto.
 - **L'Operatore $\mathbb{E}_{z \sim q(z|x)}$:** Indica il valore atteso (media statistica) calcolato su tutti i possibili vettori latenti z campionati dalla distribuzione interna $q(z|x)$ generata dall'encoder.
 - **Il Termine $\log p(x|z)$:** Rappresenta la log-verosimiglianza che il decoder ha di rigenerare l'input esatto x partendo dal codice z . Dal punto di vista pratico, si traduce nell'errore quadratico medio (MSE) per dati continui o nella *Cross-Entropy* per dati binari.
 - questa parte della formula è uguale all'**MSE** se vuoi vedere la dimostrazione di ugualianza approfondisci
- $\mathcal{D}_{\text{KL}}(q(z|x) \parallel \mathcal{N}(0, I))$ (**Penalizzazione di Kullback-Leibler**):
 - **L'Operatore $\mathcal{D}_{\text{KL}}(\cdot \parallel \cdot)$:** È una misura asimmetrica di divergenza statistica (entropia relativa) che quantifica la discrepanza informativa tra due distribuzioni di probabilità.
 - **I Membri $q(z|x)$ e $\mathcal{N}(0, I)$:** Misura quanto la gaussiana locale creata dall'encoder per una specifica immagine si scosti da una gaussiana standard centralizzata nell'origine con varianza unitaria.
 - **Funzione Regolarizzatrice:** Questo blocco agisce come un vincolo geometrico di compattazione. Impedisce alle medie μ di divergere verso l'infinito e forza le varianze σ^2 a stabilizzarsi sull'unità, garantendo l'assenza di vuoti nello spazio latente.

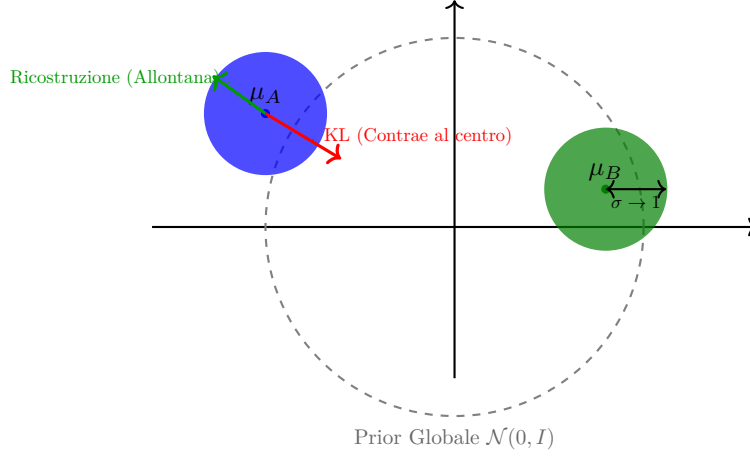


Figura 2: L'equilibrio dinamico: La Divergenza KL agisce come forza centripeta, attirando le medie verso l'origine e dilatando la varianza a 1. L'Errore di Ricostruzione funge da forza centrifuga, separando i cluster visivamente incompatibili per evitare ambiguità.

2.4 Formazione dei Cluster Non Supervisionati e Interpolazione

Un malinteso comune è ritenere che la KL riconosca le classi o che l'architettura necessiti di etichette per raggruppare immagini simili. In realtà, il clustering è una **proprietà emergente e spontanea**.

1. **Invarianza dei Pixel:** Il Decoder richiede che vettori latenti con un'alta sovrapposizione di pixel (es. due stili del numero "3") vengano mappati vicini, in modo da poter utilizzare gli stessi pesi per la ricostruzione, abbattendo la loss.
2. **La Regolarizzazione Isotropica:** La KL impone a tutte queste gaussiane di avere media quasi nulla e varianza ampia ($\sigma \approx 1$).

Essendo forzate ad occupare la medesima regione centrale ed avendo un raggio dilatato, le distribuzioni delle classi affini sono **geometricamente costrette a sovrapporsi**.

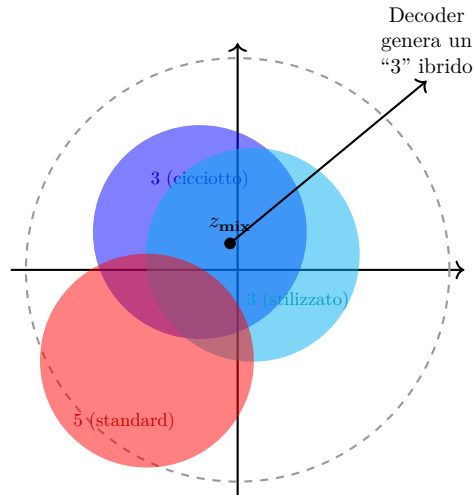


Figura 3: Spazio Latente VAE continuo (L'Effetto Mosaico). Le gaussiane non sono separate, ma incastrate. Campionando nell'area di intersezione (z_{mix}), la rete esegue un'interpolazione semantica, unendo i tratti morfologici delle distribuzioni sovrapposte.

Conclusione Analitica: Lo spazio latente del VAE cessa di essere un archivio discreto e diviene una singola matrice genetica. Campionando all'interno delle aree di sovrapposizione probabilistica, l'algoritmo non fallisce, ma esegue una **sintesi morfologica** tra i gradienti di pixel delle due distribuzioni interagenti, generando feature fluide e immagini assolutamente inedite.

ATTENZIONE: se estraiamo un punto in mezzo a due gaussiane, avremo l'incertezza sul numero da generare, essendo gaussiane a meno che non si trovi esattamente in mezzo, avremo la probabilità che tende più da una parte che dall'altra.

2.5 La Meccanica dell'Aggiornamento dei Pesi e Flusso dei Gradienti

Per comprendere come la loss variazionale $\mathcal{L}_{\text{VAE}}$ modifichi i parametri sinaptici, occorre mappare il flusso dei gradienti all'interno del grafo computazionale. L'ottimizzazione avviene simultaneamente su due fronti.

2.5.1 Il Grafo Computazionale e il Reparameterization Trick

In un'architettura variazionale, il flusso standard della backpropagation incontrerebbe una barriera stocastica non derivabile in corrispondenza del campionamento del vettore latente z . L'isolamento del rumore tramite il *Reparameterization Trick* ($z = \mu + \sigma \odot \varepsilon$) ristruttura il grafo rendendolo interamente differenziabile.

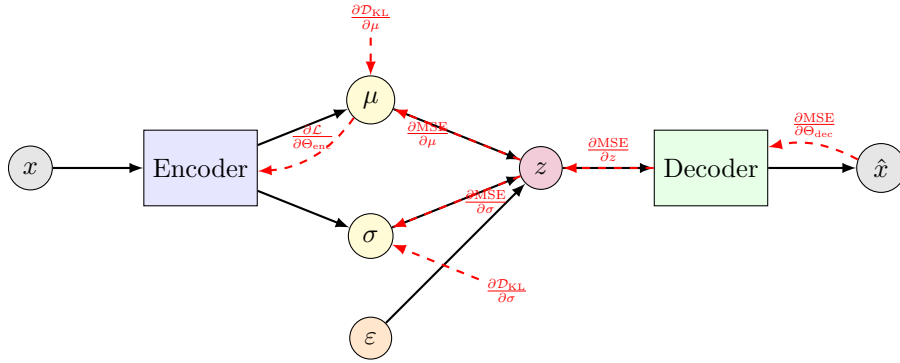


Figura 4: Flusso deterministico del gradiente (linee rosse tratteggiate). Grazie al reparameterization trick, l'errore di ricostruzione fluisce liberamente dal decoder fino all'encoder attraverso il canale continuo di z .

2.5.2 Derivazione Matematica dei Gradienti dei Parametri

La discesa del gradiente corregge l'insieme dei pesi e dei bias dell'encoder (Θ_{enc}) e del decoder (Θ_{dec}) basandosi sulla scomposizione additiva della loss totale $\mathcal{L}_{\text{total}} = \text{MSE} + \mathcal{D}_{\text{KL}}$.

1. **Aggiornamento del Decoder (Θ_{dec}):** Il blocco di regolarizzazione statistica $\mathcal{D}_{\text{KL}}$ è calcolato a monte del campionamento e non dipende dai parametri del decoder. Pertanto, la derivata parziale rispetto ai pesi W_{dec} vede il termine della KL come una costante nulla:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W_{\text{dec}}} = \frac{\partial \text{MSE}}{\partial W_{\text{dec}}} + \frac{\partial \mathcal{D}_{\text{KL}}}{\partial W_{\text{dec}}} = \frac{\partial \text{MSE}}{\partial W_{\text{dec}}} + 0 \quad (9)$$

La correzione dei pesi del decoder segue l'algoritmo classico di ottimizzazione stocastica regolata dal learning rate η :

$$W_{\text{dec}}^{(t+1)} = W_{\text{dec}}^{(t)} - \eta \cdot \frac{\partial \text{MSE}}{\partial W_{\text{dec}}^{(t)}} \quad (10)$$

2. **Aggiornamento dell'Encoder (Θ_{enc}):** L'encoder controlla la forma dello spazio latente, influenzando direttamente sia la disposizione geometrica delle medie e delle varianze (sotto la giurisdizione della KL), sia il punto campionato z che determina la bontà di ricostruzione (MSE). Il gradiente totale rispetto a W_{enc} è la combinazione lineare delle due forze:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W_{\text{enc}}} = \frac{\partial \text{MSE}}{\partial W_{\text{enc}}} + \frac{\partial \mathcal{D}_{\text{KL}}}{\partial W_{\text{enc}}} \quad (11)$$

Applicando la regola della catena sul primo termine ed esplicitando la forma chiusa della KL derivata rispetto ai parametri distribuzionali locali (μ_j, σ_j^2), otteniamo le componenti esatte del gradiente:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W_{\text{enc}}} = \sum_{j=1}^K \left(\frac{\partial \text{MSE}}{\partial z_j} \cdot \frac{\partial z_j}{\partial W_{\text{enc}}} \right) + \frac{\partial}{\partial W_{\text{enc}}} \left[-\frac{1}{2} \sum_{j=1}^K (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2) \right] \quad (12)$$

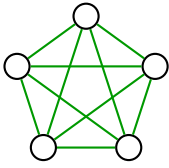
2.5.3 Il Bilancio Dinamico delle Forze sui Neuroni dell'Encoder

- **Il gradiente della KL** genera una spinta proporzionale a $-\mu_j$ e a $(1 - \sigma_j^2)$. Se una media si allontana dallo zero o una varianza collassa, questa derivata costringe i pesi dell'encoder a modificare la propria configurazione per ritrarre la distribuzione verso la forma sferica della prior.
- **Il gradiente dell'MSE** si propaga a ritroso attraverso le matrici del decoder, quantifica l'errore sui pixel e colpisce i parametri dell'encoder intimando di spostare le medie μ di classi dissimili in direzioni opposte, contrastando la compressione della KL per preservare la separabilità semantica.

L'aggiornamento finale dell'encoder avviene accumulando i contributi dei due vettori di forza:

$$W_{\text{enc}}^{(t+1)} = W_{\text{enc}}^{(t)} - \eta \cdot \left(\frac{\partial \text{MSE}}{\partial W_{\text{enc}}^{(t)}} + \frac{\partial \mathcal{D}_{\text{KL}}}{\partial W_{\text{enc}}^{(t)}} \right) \quad (13)$$

3 reti di hopfield



- reti composte da un singolo strato interconnesso
- connessioni simmetriche: $W_{ji} = W_{ij}$
- neuroni binari: $\{0, 1\}$ attivo/non attivo oppure $\{-1, 1\}$ funzioni usate per simmetria

Le reti di **hopfield** sono in grado di memorizzare varie configurazioni di neuroni e sono molto utili per ricostruire qualcosa partendo da informazioni frammentate, per capire meglio immaginiamo un sottoinsieme di neuroni correttamente attivati, essi esercitano una spinta attrattiva sulle unità corrotte o silenziose (forzandole a cambiare stato per conformarsi al pattern memorizzato).

3.1 Aggiornamento dei pesi:

ad ogni ciclo vengono aggiornati tutti i neuroni ma uno alla volta in ordine casuale:

- input neurone: $z = \sum_{j \neq i} W_{ji} Y_j(t) - b_i$

sommatoria di tutti gli stati dei neuroni associati al loro peso al tempo t . b_i = bias sottratto per favorire l'inattività ad attivarsi meno facilmente.

3.2 Dinamica di Transizione degli Stati Locali

La regola di attivazione definisce il comportamento di un neurone i -esimo in base al potenziale post-sinaptico netto ricevuto al tempo t :

$$Y_i(t+1) = \begin{cases} 1 & \text{se } \sum_{j \neq i} W_{ji} Y_j(t) - b_i > 0 \\ 0 & \text{se } \sum_{j \neq i} W_{ji} Y_j(t) - b_i \leq 0 \end{cases} \quad (14)$$

Per comprendere l'evoluzione microscopica del sistema, definiamo lo stimolo netto come $I_i(t) = \sum_{j \neq i} W_{ji} Y_j(t) - b_i$. Possiamo mappare le quattro transizioni logiche fondamentali che governano la convergenza della memoria associativa:

- **Conservazione dello Stato Attivo (Stabilità):** Se $Y_i(t) = 1$ e lo stimolo netto $I_i(t) > 0 \rightarrow Y_i(t+1) = 1$. Il neurone conferma la propria attivazione poiché coerente con il contesto dei pesi.
- **Conservazione dello Stato Silente (Inattività):** Se $Y_i(t) = 0$ e lo stimolo netto $I_i(t) \leq 0 \rightarrow Y_i(t+1) = 0$. Il neurone rimane spento, preservando il pattern di vuoto informativo.
- **Inibizione Dinamica (Correzione del Rumore):** Se $Y_i(t) = 1$ ma lo stimolo netto $I_i(t) \leq 0 \rightarrow Y_i(t+1) = 0$. Il neurone si trovava in uno stato di falsa attivazione dovuto al rumore; la pressione collettiva ne impone lo spegnimento.

- **Eccitazione Dinamica (Completamento del Pattern):** Se $Y_i(t) = 0$ ma lo stimolo netto $I_i(t) > 0 \rightarrow Y_i(t+1) = 1$. Il neurone sperimenta un'accensione forzata indotta dai neuroni adiacenti per ricostruire la frazione mancante del ricordo.

3.3 funzione di energia: (vale per le reti simmetriche $\{-1, 1\}$)

ci serve a capire quanta energia ha la rete (quanto ancora non è una rete stabile). inoltre dato che è sempre decrescente, garantisce la convergenza.

$$E(Y) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N W_{ij} Y_i Y_j + \sum_{i=1}^N b_i Y_i$$

- **$E(Y)$ (Energia Globale del Sistema):** È una funzione scalare limitata inferiormente. Funziona come una “mappa topografica”: i pattern che la rete deve memorizzare corrispondono ai punti più bassi di questa superficie (valli o bacini di attrazione). Più il valore è basso, più la configurazione corrente dei neuroni è vicina a un ricordo stabile.

- **Il Fattore di Scala $-\frac{1}{2}$:**

- **Il Segno Meno $(-)$:** Stabilisce il segno dell'interazione. Se due neuroni sono legati da un peso positivo ($W_{ij} > 0$) e sono entrambi accesi ($Y_i = 1, Y_j = 1$), il loro prodotto è positivo. Il segno meno davanti trasforma questa stabilità in un valore di energia negativo, abbassando il costo globale e premiando la coerenza del pattern.
- **Il Coefficiente $\frac{1}{2}$:** Compensa la doppia scansione della doppia sommatoria. Poiché la rete ha connessioni simmetriche ($W_{ij} = W_{ji}$), il legame tra il neurone i e il neurone j viene conteggiato due volte (una volta come coppia (i, j) e una come (j, i)). Moltiplicare per la metà serve a normalizzare il valore, calcolando l'energia effettiva di ogni singola sinapsi senza duplicazioni.

- $\sum_{i=1}^N \sum_{j=1}^N W_{ij} Y_i Y_j$ (**Termine di Interazione Mutua o Quadratica**):

- **La Doppia Sommatoria:** Esegue una scansione combinatoria completa di tutte le possibili coppie di neuroni presenti nella rete (da 1 a N).
- **Il Prodotto $W_{ij} Y_i Y_j$:** Rappresenta l'energia interna della connessione. Se il peso concorda con lo stato dei neuroni (es. peso positivo e neuroni entrambi accesi, oppure peso negativo e un neurone spento), il sistema sperimenta una contrazione energetica locale. Se invece c'è disaccordo (es. neuroni entrambi accesi ma uniti da un legame inibitorio negativo), l'energia sale, indicando instabilità strutturale.

- $\sum_{i=1}^N b_i Y_i$ (**Termine di Potenziale Lineare o Contributo dei Bias**):

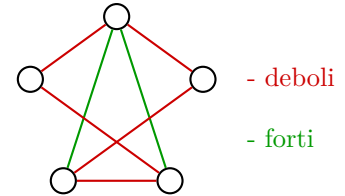
- **Il Vettore b_i :** Rappresenta la soglia di attivazione interna (*bias*) isolata per ogni singolo neurone. Agisce come un campo magnetico esterno indipendente dall'interazione tra i nodi.
- **La Sommatoria Lineare:** Calcola il costo di attivazione energetica statico. Poiché questo termine viene sommato (a differenza del blocco quadratico preceduto dal meno), la presenza di un bias positivo $b_i > 0$ associato a un neurone acceso ($Y_i = 1$) fa aumentare linearmente l'energia complessiva. Di conseguenza, il bias opera come una forza frenante o resistenziale, rendendo l'attivazione di quel neurone intrinsecamente più dispendiosa e favorendo l'inattività del sistema a meno di forti spinte sinaptiche contrarie.

3.4 apprendimento: (hebbian rule)

non si usa la classica backprop, bensì la regola di hebb.

$$W_{ji} = Y_j Y_i \quad \text{regola base (valida per un solo pattern)}$$

questo implica che se 2 neuroni hanno la stessa attivazione il loro peso sarà positivo (eccitatorio) altrimenti sarà negativo (inibitorio).



regola generale (più pattern): *(intesi come i vari minimi locali di una funzione)*

$$W_{ij} = \frac{1}{N} \sum_{p \in \{1, P\}} Y_i^p Y_j^p$$

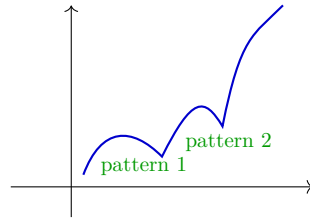
- **Il Coefficiente di Normalizzazione $\frac{1}{N}$:**
 - **Il Termine N :** Rappresenta il numero totale di neuroni che compongono l'intera architettura della rete.
 - **Funzione di Scala:** Divide la somma per il numero di nodi per evitare che i valori dei pesi sinaptici crescano a dismisura all'aumentare della dimensione della rete. Mantiene i pesi in un range numerico stabile, garantendo che i gradienti energetici rimangano proporzionati e non mandino il sistema in saturazione.
- $\sum_{p \in \{1, P\}}$ (**Operatore di Accumulazione dei Ricordi**):
 - **Il Set $\{1, P\}$:** Rappresenta l'insieme di tutti i P pattern fondamentali (immagini, vettori, configurazioni ideali) che la rete deve memorizzare contemporaneamente.
 - **La Sommatoria:** Implementa il principio della sovrapposizione dei ricordi. La rete non impara un pattern alla volta cancellando il precedente; al contrario, i contributi di tutti i P vettori vengono **sommati linearmente** dentro la stessa identica matrice dei pesi W . Ogni sinapsi accumula la storia delle co-attivazioni di tutti i pattern del dataset.
- **Il Prodotto $Y_i^p Y_j^p$ (Correlazione Hebbiana al Pattern p):**
 - **I Termini Y_i^p e Y_j^p :** Rappresentano lo stato obiettivo (acceso o spento) del neurone i e del neurone j specificamente all'interno del pattern p -esimo.
 - **Logica del Prodotto (Se gli stati sono binarizzati in $\{-1, 1\}$):** Se nel pattern p i due neuroni hanno lo stesso stato (entrambi 1 o entrambi -1), il loro prodotto è $+1$, quindi il peso W_{ij} aumenta (cooperazione). Se hanno stati opposti, il loro prodotto è -1 , quindi il peso diminuisce (inibizione).
 - **Logica del Prodotto (Se gli stati sono binarizzati in $\{0, 1\}$):** Il prodotto restituisce 1 solo se entrambi i neuroni sono attivi ($1 \cdot 1 = 1$). Se uno dei due o entrambi sono spenti (0), il prodotto si annulla (0). Questo significa che il legame sinaptico si rinforza attivamente solo per stimolare l'effetto di co-accensione delle unità, rispecchiando fedelmente il postulato biologico di Hebb: *“Neuroni che sparano insieme, si legano insieme”*.

ad esempio con 3 pattern:

$$W_{ij} = \frac{1}{N} \sum_{p=1}^3 Y_i^p Y_j^p = \frac{1}{N} (Y_i^1 Y_j^1 + Y_i^2 Y_j^2 + Y_i^3 Y_j^3)$$

Tutti e tre i pattern (il pattern 1, il pattern 2 e il pattern 3) vengono "compressi" e fusi insieme dentro un'unica grande matrice di pesi W .

Quando la matrice W ha accumulato i 3 pattern tramite quella somma, la superficie dell'energia globale della rete è stata deformata. Immagina la superficie dell'energia come un foglio di gomma teso: la formula hebbiana ha scavato tre grandi conche (valli) in corrispondenza dei tre pattern puliti. Queste valli si chiamano mathematically Attrattori Stabili.



Inserendo un pattern con del rumore, con poche iterazioni l'energia si stabilizza portandoci in una conca (pattern memorizzato).

Se ci sono troppi pattern diventa difficile memorizzarli. soluzione: alzare l'energia della curva per far evidenziare i pattern.